

Programación 3

Trabajo Práctico

Lista de Compras con Autenticación

React · TypeScript · React Router · Zustand · React Query · json-server

1. Objetivos

Al finalizar este trabajo el alumno deberá ser capaz de:

- Organizar un proyecto React con **estructura de módulos**.
- Implementar **protección de rutas** con React Router v6.
- Gestionar estado global de autenticación con **Zustand**.
- Consumir una API REST con **React Query** (useQuery + useMutation).
- Simular una API REST con **json-server**.

2. Descripción del trabajo

Se debe desarrollar una aplicación web de lista de compras que cumpla con las siguientes condiciones:

- Solo puede acceder un usuario autenticado. Las credenciales están hardcodeadas en el store de Zustand.
- Si el usuario intenta acceder a /shopping-list sin estar autenticado, es redirigido automáticamente a /login.
- Una vez autenticado, el usuario puede ver, agregar, marcar como comprado y eliminar productos de la lista.
- Los productos se persisten en json-server (archivo db.json) y se consultan mediante React Query.

Credenciales del usuario hardcodeado

Email: pepe@gmail.com

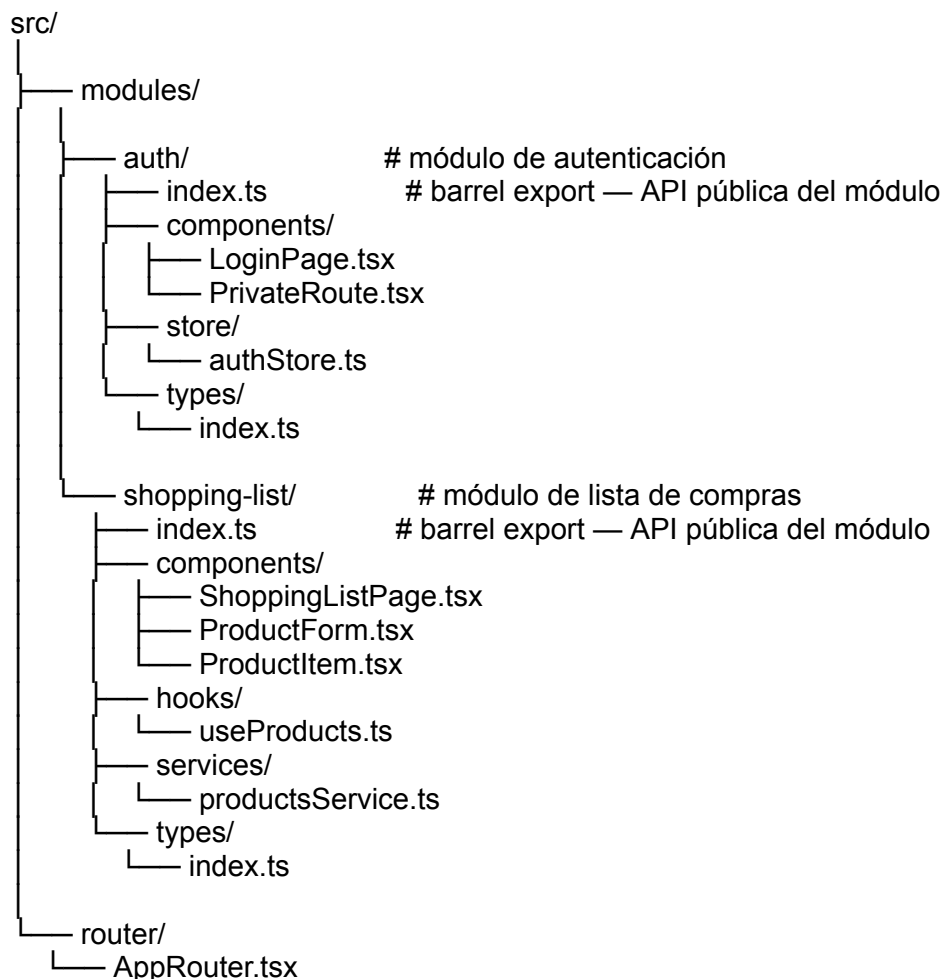
Contraseña: pepePassword23

Estas credenciales deben vivir dentro del authStore (Zustand), no en el componente.

3. Tecnologías obligatorias

Tecnología	Uso esperado
React + TypeScript	Base del proyecto. Todos los componentes deben usar tipado explícito.
React Router v6	Manejo de rutas. Se debe usar <Outlet /> y rutas anidadas para la protección.
Zustand	Estado global de autenticación: user, isAuthenticated, login(), logout().
React Query v5	GET de productos con useQuery. POST/PATCH/DELETE con useMutation + invalidateQueries.
json-server	Simula la API REST. Corre en el puerto 3001 y persiste los datos en db.json.
Vite	Entorno de desarrollo. El proyecto se inicializa con Vite + React + TS.

4. Estructura de carpetas



5. Consignas

5.1 Módulo auth

- a) En `authStore.ts`, crear un store de Zustand con el siguiente estado y acciones:
- **Estado:** `user: User | null, isAuthenticated: boolean, error: string | null`
 - **Acciones:** `login(credentials): boolean` → devuelve true si las credenciales son correctas, false si no.
 - **Acciones:** `logout(): void` → resetea el estado.
- b) En `PrivateRoute.tsx`, implementar la protección de rutas:
- Leer `isAuthenticated` del store.
 - Si es `false`, redirigir a `/login` usando `<Navigate replace />`.
 - Si es `true`, renderizar `<Outlet />` para mostrar la ruta hija.
- c) En `LoginPage.tsx`, crear el formulario de login. Al hacer submit:
- Llamar a `login()` del store con el email y la contraseña del formulario.
 - Si `login()` devuelve true, navegar a `/shopping-list` con `useNavigate`.
 - Si devuelve false, mostrar el mensaje de error que dejó el store.

5.2 Módulo shopping-list

- a) En `productsService.ts`, implementar las funciones que se comunican con json-server:
- `getProducts()` → GET `/products`
 - `createProduct(product)` → POST `/products`
 - `updateProduct(id, changes)` → PATCH `/products/:id`
 - `deleteProduct(id)` → DELETE `/products/:id`
- b) En `useProducts.ts`, crear los custom hooks de React Query:
- `useProducts()` → `useQuery` sobre `getProducts`.
 - `useCreateProduct()` → `useMutation` con invalidación de caché en `onSuccess`.
 - `useToggleDone()` → `useMutation` para PATCH del campo `done`.
 - `useDeleteProduct()` → `useMutation` para DELETE.
- c) En `ShoppingListPage.tsx`, mostrar la lista separada en dos secciones: pendientes y comprados.
- d) En `ProductItem.tsx`, implementar el checkbox (toggle done) y el botón eliminar.
- e) En `ProductForm.tsx`, implementar el formulario con los campos: nombre, cantidad y unidad.

5.3 Router

En `AppRouter.tsx` configurar las rutas:

- `/login` → ruta pública.
- `/shopping-list` → ruta privada, envuelta en `PrivateRoute`.
- `/*` → redirige a `/shopping-list`.

5.4 Providers en App.tsx

Envolver `AppRouter` con `QueryClientProvider`. El `queryClient` debe crearse una sola vez fuera del componente.

5.5 json-server

El archivo `db.json` debe tener una colección `products` con al menos 3 productos de ejemplo. Cada producto debe tener los campos:

- `id` (number, generado por `json-server`)
- `name` (string)
- `quantity` (number)
- `unit` (string — ej: "kg", "litros", "unidades")
- `done` (boolean)

⚡ Scripts sugeridos en package.json

"dev": "vite" → corre el frontend

"api": "json-server --watch db.json --port 3001"

"start": "concurrently \"npm run api\" \"npm run dev\""

Con `npm run start` se levantan ambos servidores a la vez.

6. Flujo esperado de la aplicación

1. El usuario ingresa a cualquier ruta (ej: `/`).
2. El router redirige a `/shopping-list`.
3. `PrivateRoute` detecta que `isAuthenticated` es `false` y redirige a `/login`.
4. El usuario completa el formulario con las credenciales correctas.
5. `login()` valida las credenciales y actualiza el store: `isAuthenticated = true`.
6. `useNavigate` lleva al usuario a `/shopping-list`.
7. `PrivateRoute` ahora permite el acceso y renderiza `<Outlet />`.
8. `ShoppingListPage` hace `useProducts()` → `React Query` llama a `getProducts()` → `json-server` responde.
9. El usuario puede agregar, marcar o eliminar productos. Cada mutación invalida la caché y `React Query` `refetch` automáticamente.
10. El botón "Salir" llama a `logout()`, resetea el store y el usuario vuelve a `/login`.